# Reviewer Pack — Minimal Number of Galois Automorphisms vs. Circuit Weights

Entry 653fdc1beeee · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-22 19:13:47 UTC

## Minimal Number of Galois Automorphisms vs. Circuit Weights

**Entry ID**: `653fdc1beeee` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-22 19:13:47 UTC

### 1. Conjecture

**Field A** (mathematical branch): Galois Theory (Automorphisms) **Field B** (complexity object): Boolean Circuit Complexity

**Statement**:

> ['For every Boolean circuit C with n inputs and output size m, there exists a Galois extension K of the finite field F2 such that the number of Galois automorphisms of K fixing the value of C is at most α(n, m), where α(n, m) = O(f(n)) for some polynomial f(n).', "For all circuits C with n inputs and output size m, the minimal number of Galois automorphisms that preserve the circuit's behavior is bounded by a polynomial in the number of inputs and outputs.", 'A counterexample to this conjecture would be a Boolean circuit with a small number of inputs and outputs whose minimal number of Galois automorphisms exceeds a polynomial bound.']

**Rationale (proposer's reasoning)**:

> ['Galois theory studies symmetries of algebraic structures, and its connections to the structure of circuits could provide insights into computational complexity.', 'Understanding the minimal symmetry of circuits might expose new techniques for proving lower bounds on circuit size or complexity.', 'This connection has not been extensively explored in complexity theory, making it a promising area for novel conjectures.']

**Taxonomy category**: `SOS_DEGREE` (status at proposal time: )

### 2. Pre-registration (Popper-style)

**Hash (SHA-256 prefix)**: `84a37bd85a2d5d0a`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

**Acceptance criterion**:

> The conjecture is supported if for all generated Boolean circuits with n inputs and m outputs, the number of Galois automorphisms that preserve circuit behavior is within a polynomial bound α(n, m) = O(f(n)), where f(n) is a polynomial. The conjecture is falsified if there exists at least one circuit with n ≤ 40 inputs and m outputs such that the minimal number of Galois automorphisms exceeds a polynomial bound.

## 3. Barrier filter (F1)

**Final**: PASS

| Barrier | Verdict | Confidence | LLM₁ | LLM₂ |
|---|---|---|---|---|
| RELATIVIZATION | UNCERTAIN | 1.00 | HITS | UNCERTAIN |
| NATURAL_PROOFS | SAFE | 0.50 | UNCERTAIN | UNCERTAIN |
| ALGEBRIZATION | UNCERTAIN | 0.90 | UNCERTAIN | HITS |
| KARP_LIPTON | SAFE | 0.50 | UNCERTAIN | UNCERTAIN |

## 4. Novelty audit

**Verdict**: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

## 5. Empirical test harness

**Multi-seed protocol**: 5 seeds (11, 23, 37, 53, 71)
**Sandbox**: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=124, elapsed=240.4s

### 5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_circuit(n, m):
        # Generate a random Boolean circuit with n inputs and m outputs
        circuit = [[random.choice([0, 1]) for _ in range(m)] for _ in range(2**n)]
        return circuit

    def galois_group_size(n):
        # Calculate the size of the Galois group for F_{2^n}
        return 2**n

    def count_galois_automorphisms(circuit, n):
        # Count the number of Galois automorphisms that preserve the circuit
        count = 0
        galois_size = galois_group_size(n)
        for i in range(galois_size):
            if all(circuit[j] == circuit[(j + i) % galois_size] for j in range(2**n)):
                count += 1
        return count

    n = random.randint(5, 40)
    m = random.randint(1, n)
    circuit = generate_boolean_circuit(n, m)

    metric_value = count_galois_automorphisms(circuit, n)
    instances_tested = 1
    conjecture_holds = True if metric_value <= (n + m)**2 else False
    counterexample = "" if conjecture_holds else f"Counterexample with n={n}, m={m}"
```

```
    return {
        "metric_name": "Number of Galois Automorphisms",
        "metric_value": metric_value,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [2**i + 1 for i in range(5, 8)]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_metric_value = sum(result["metric_value"] for result in results) / len(results)
    std_metric_value = math.sqrt(sum((result["metric_value"] - mean_metric_value)**2 for result in
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample=\"First failing seed\" first_failing_seed={first_
    else:
        print("RESULT: INCONCLUSIVE reason=unknown")
```

## 6. Per-seed results

*(no seed-level data — test crashed before producing TRIAL: lines)*

## 7. Test stdout (last 2KB)

```
TIMEOUT after 240s
```

## 8. Critic adversarial review

**Critic verdict**: CHALLENGE

**Critic reasoning**:

> skipped: test crashed before producing data

## 9. Final verdict & safety rail

**Verdict**: INCONCLUSIVE

**Reasoning**:

> The test timed out before producing data, which prevents us from verifying whether the number of Galois automorphisms that preserve circuit behavior is within a polynomial bound. | next: Run the test again with increased time limits to ensure it completes and produces results.

## 11. Audit log (LLM calls)

**Total LLM calls**: 9

| # | Phase | Provider | Model | Tokens in | out | Latency (ms) |
|---|---|---|---|---|---|---|
| 1 | propose | ollama_remote | glm4:latest | 0 | 0 | 14541 |
| 2 | preregistration | ollama_remote | glm4:latest | 0 | 0 | 10301 |
| 3 | novelty | ollama_remote | glm4:latest | 0 | 0 | 8343 |
| 4 | novelty | ollama_remote | glm4:latest | 0 | 0 | 8559 |
| 5 | test_gen | ollama_remote | qwen2.5-coder:7b | 0 | 0 | 13189 |
| 6 | test_gen | ollama_remote | qwen2.5-coder:7b | 0 | 0 | 8002 |
| 7 | test_gen | ollama_remote | qwen2.5-coder:7b | 0 | 0 | 8902 |
| 8 | test_gen | ollama_remote | qwen2.5-coder:7b | 0 | 0 | 7916 |
| 9 | judge | ollama_remote | glm4:latest | 0 | 0 | 14616 |

**Totals**: 0 input tokens, 0 output tokens, ~$0.0000 cost, 94370 ms total latency. Provider mix: {'ollama_remote': 9}

*(full prompt+response transcripts available in `research/audit/653fdc1beeee.jsonl`)*

## 12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/653fdc1beeee.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

**Sandbox file**: `research/pvsnp_sandbox/test_*.py` (per-cycle)
**Replay tarball**: `research/replay/653fdc1beeee.tar.gz` (if generated)